

Question 1:

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat program1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);

        printf("Command execution failed\n");
    }
    else {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }
}
```

```
    printf("Command execution failed\n");
}
else {
    printf("Parent PID: %d\n", getpid());
    wait(NULL);
    printf("Child process completed.\n");
}

return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc program1.c -o program1
himasree@LAPTOP-41COT5AA:~/OSLab$ ./program1
Enter Linux command: ls
Parent PID: 403
Child PID: 404
MyProject exec exp1.c exp2.c exp3.c exp4.c exp5.c exp6.c exp7.c exp8.c exp9.c program.c program1.c
MyProject exp1 exp2 exp3 exp4 exp5 exp6 exp7 exp8 exp9 program program1
Child process completed.
```

Question 2:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("sample.txt", O_RDONLY);

    if (source == -1)
    {
        printf("Error opening source file.\n");
        return 1;
    }

    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination == -1)
    {
        printf("Error creating destination file.\n");
        close(source);
        return 1;
    }
}
```



```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(5);
    }
    else {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        sleep(10);
    }

    return 0;
}

```

Question 4:

```

himasree@LAPTOP-41COT5AA:~/OSLab$ nano program4.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc program4.c -o program4
himasree@LAPTOP-41COT5AA:~/OSLab$ ./program4
Child 1 PID: 906
Child 2 PID: 907
First child finished using wait().
Second child finished using waitpid().
Parent process completed.

```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t p1, p2;

    p1 = fork();

    if (p1 == 0) {
        printf("Child 1 PID: %d\n", getpid());
        sleep(2);
        return 0;
    }

    p2 = fork();

    if (p2 == 0) {
        printf("Child 2 PID: %d\n", getpid());
        sleep(4);
        return 0;
    }

    wait(NULL);
```

```
wait(NULL);
printf("First child finished using wait().\n");

waitpid(p2, NULL, 0);
printf("Second child finished using waitpid().\n");

printf("Parent process completed.\n");

return 0;
```